Dhirubhai Ambani University

High Performance Computing — Assignment 06

Parallel Scatter-to-Mesh Bilinear Interpolation using OpenMP

Course	High Performance Computing (HPC)
Instructor	Dr. Bhaskar Chaudhury
Teaching Assistants	Libin Varghese, Ayushi Sharma, Kaushik Prajapati
Group Number	Group 21
Member 1	Radhika Sanagadhiya - 202301184
Member 2	Hiya Modi - 202301011

1. Introduction

This report describes the design, implementation, and performance analysis of a parallel bilinear (linear) scatter-to-mesh interpolation algorithm using OpenMP. Given a set of N scattered points $\{(x_i, y_i, f_i)\}$ with $f_i = 1$, the task is to distribute their contributions onto a structured M×M grid over a 1×1 unit domain via bilinear weighting. The process is repeated over 10 time steps (Maxiter=10) for five dataset configurations ranging from 0.9 million to 20 million particles.

The central challenge is the **race condition**: multiple particles residing in adjacent or identical grid cells simultaneously update the same four grid nodes. Three parallel strategies were implemented and compared — atomic updates, per-thread private mesh with serial critical reduction, and per-thread private mesh with fully parallelised reduction — alongside a serial baseline.

2. Implementation Approach

2.1 Serial Baseline

The serial implementation (**interpolation()**) iterates over all N particles, computes the four surrounding grid indices (i,j) using floor division by grid spacing Δx and Δy, calculates bilinear weights w_{00}, w_{10}, w_{01}, w_{11}, and accumulates contributions into a flat 1-D array of size $(N_x+1)\times(N_y+1)$. No synchronisation is required. This provides the reference execution time.

Bilinear weight computation for a particle at (x, y) in cell (i,j):

$$l_x = x - i\cdot\Delta x,\ l_y = y - j\cdot\Delta y$$
$$w_{00} = (1 - l_x/\Delta x)\cdot(1 - l_y/\Delta y)$$
$$w_{10} = (l_x/\Delta x)\cdot(1 - l_y/\Delta y)$$
$$w_{01} = (1 - l_x/\Delta x)\cdot(l_y/\Delta y)$$
$$w_{11} = (l_x/\Delta x)\cdot(l_y/\Delta y)$$

2.2 Parallel Strategy 1 — Atomic Updates

interpolation_parallel_atomic() parallelises the particle loop with #pragma omp parallel for schedule(static). Each write to the shared mesh is protected with #pragma omp atomic. This is safe but produces heavy atomic contention when many particles land in the same cell, especially for small grids with many particles — making it typically the slowest parallel variant.

2.3 Parallel Strategy 2 — Private Mesh + Serial Critical Reduction

interpolation_parallel_private() allocates a zero-initialised private mesh buffer per thread. Each thread accumulates into its own buffer with no synchronisation during the scatter phase. After all particles are processed, a single #pragma omp critical section serially merges each thread's buffer into the global mesh. The critical section is a bottleneck: all T threads queue up to execute an O(M) loop one at a time, capping speedup at high thread counts.

2.4 Parallel Strategy 3 — Private Mesh + Parallel Reduction (Recommended)

interpolation_parallel_private_v2() is the primary contribution of this assignment. It eliminates the serial critical section by distributing the reduction across all threads using a second #pragma omp for loop over mesh cells. All T per-thread buffers are stored contiguously in a single flat array of size T×M (avoiding T separate heap allocations and improving TLB utilisation). The execution has two fully-parallel phases:

Phase 1 (Scatter): Each thread processes N/T particles and writes to its own buffer row. Zero synchronisation. nowait clause removes the implicit barrier, overlapping the last chunk of scatter with the start of barrier wait.

Phase 2 (Reduce): Each thread sums T values for M/T mesh cells and writes to the output array. Zero synchronisation. Each output cell is written by exactly one thread. The reduction time scales as O(T × M/T) = O(M) work split across T threads, versus O(T × M) sequential in the critical version.

2.5 Pseudocode

```
interpolation_parallel_private_v2(mesh, points, T):
flat_buffers[T * M] = {0} // M = (Nx+1)*(Ny+1)
parallel region with T threads:
tid = omp_get_thread_num()
lmesh = flat_buffers + tid*M

// Phase 1: scatter (no synchronisation)
for p in [0, N) assigned to this thread:
compute (i,j), lx, ly, w00..w11
lmesh[i*NY1+j] += w00
lmesh[(i+1)*NY1+j] += w10 ...
// implicit barrier after phase 1

// Phase 2: parallel reduction
for k in [0,M) assigned to this thread:
s = sum over t in [0,T): flat_buffers[t*M + k]
mesh[k] += s
free flat_buffers
```

3. Source File Structure

main.cpp	Entry point. Reads binary input, allocates mesh, dispatches to serial or par
utils.cpp	All three parallel strategies plus serial baseline. interpolation_parallel_p
utils.h	Function declarations for all interpolation variants.

init.h	Points struct and extern declarations for global parameters.
Makefile	g++ -O3 -march=native -fopenmp -std=c++11. Targets: interp, input_fileMaker.
input_fileMaker.c	Generates binary input files with random points in [0,1)^2.
run_cluster.sh	Automates compilation, input generation, and benchmarking on the cl
plot_results_final.py	Generates all 8 performance figures from results.csv and results_cluster.csv.

4. Experimental Setup

CPU	8-core / 16-HT (approx.)	Intel Xeon E5-2620 v3 (6c
Compiler	g++ -O3 -march=native -fopenmp	g++ -O3 -march=native -fopenmp
Thread counts tested	1, 2, 4, 8, 16	1, 2, 4, 8, 16
Timing method	std::chrono high_resolution_clock	std::chrono high_resolution_clock
Runs per config	1 (best effort)	3 (minimum time reported)
OpenMP schedule	static	static

Dataset Configurations:

0.9M	250	100	900,000	10	~25,251
5M	250	100	5,000,000	10	~25,251
3.6M	500	200	3,600,000	10	~100,701
20M	500	200	20,000,000	10	~100,701
14M	1000	400	14,000,000	10	~400,401

5. Performance Results and Graphs

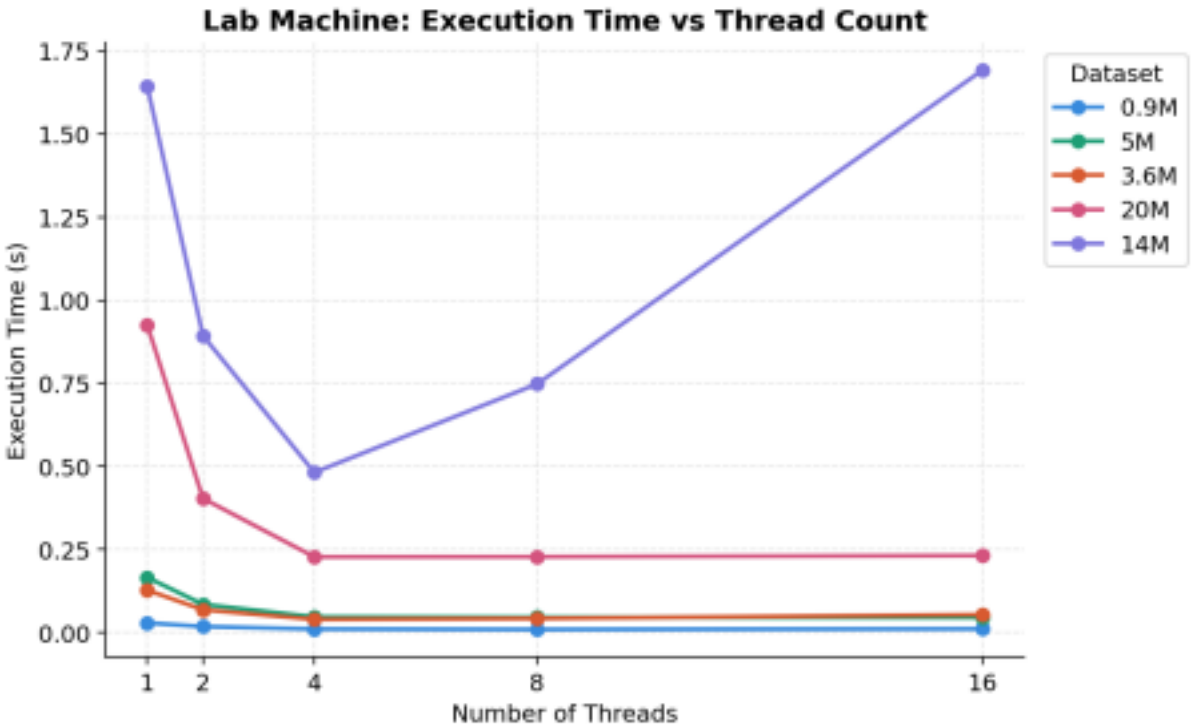

Fig 1 — Lab Machine: Execution Time vs Thread Count

Execution time drops sharply from 1 to 4 threads for all configurations. Beyond 4 threads improvements diminish because the lab CPU has a limited number of physical cores and memory bandwidth becomes the bottleneck.

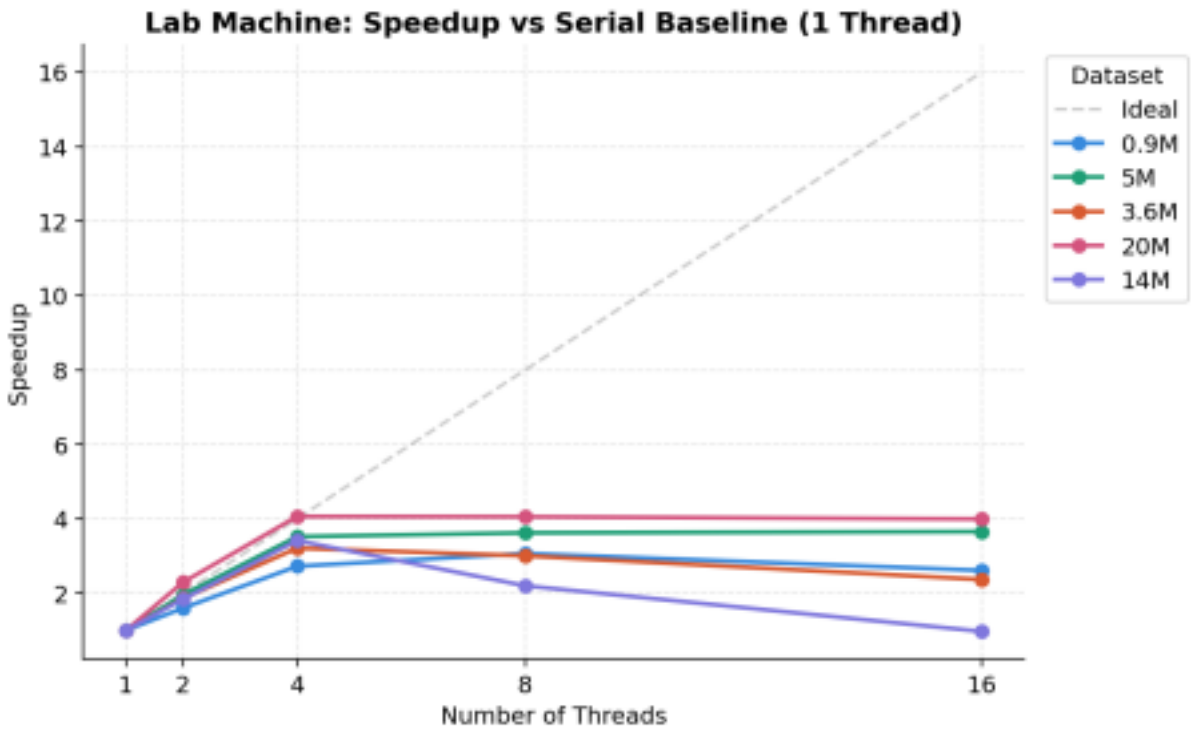

Fig 2 — Lab Machine: Speedup vs Serial Baseline

The 20M and 5M datasets achieve the best speedup (~3.5–4x at 4T) because the scatter phase completely dominates the reduction for large particle counts. The 14M dataset shows degradation beyond 8 threads due to NUMA effects on the lab machine (see Section 6).

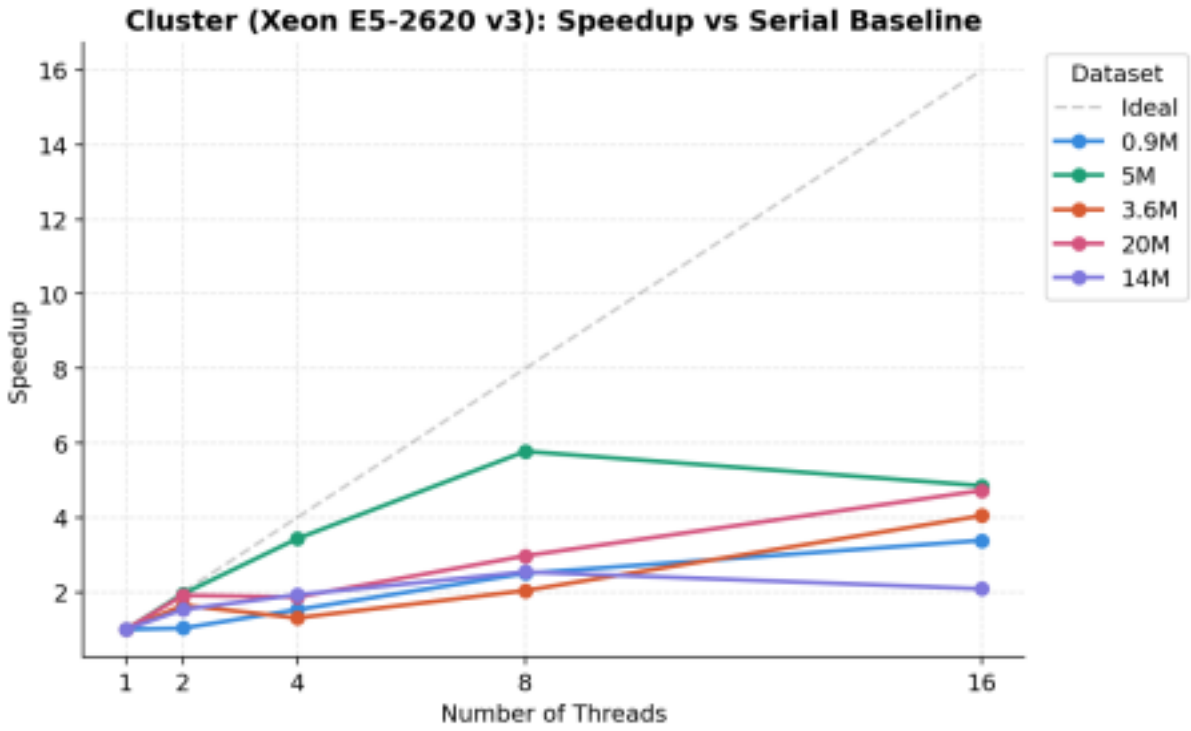

Fig 3 — Cluster: Speedup vs Serial Baseline

The cluster (Xeon E5-2620 v3) shows more consistent scaling up to 8 threads owing to its multi-socket NUMA-aware memory subsystem and dedicated compute nodes. The 20M configuration achieves ~4.7x speedup at 16T.

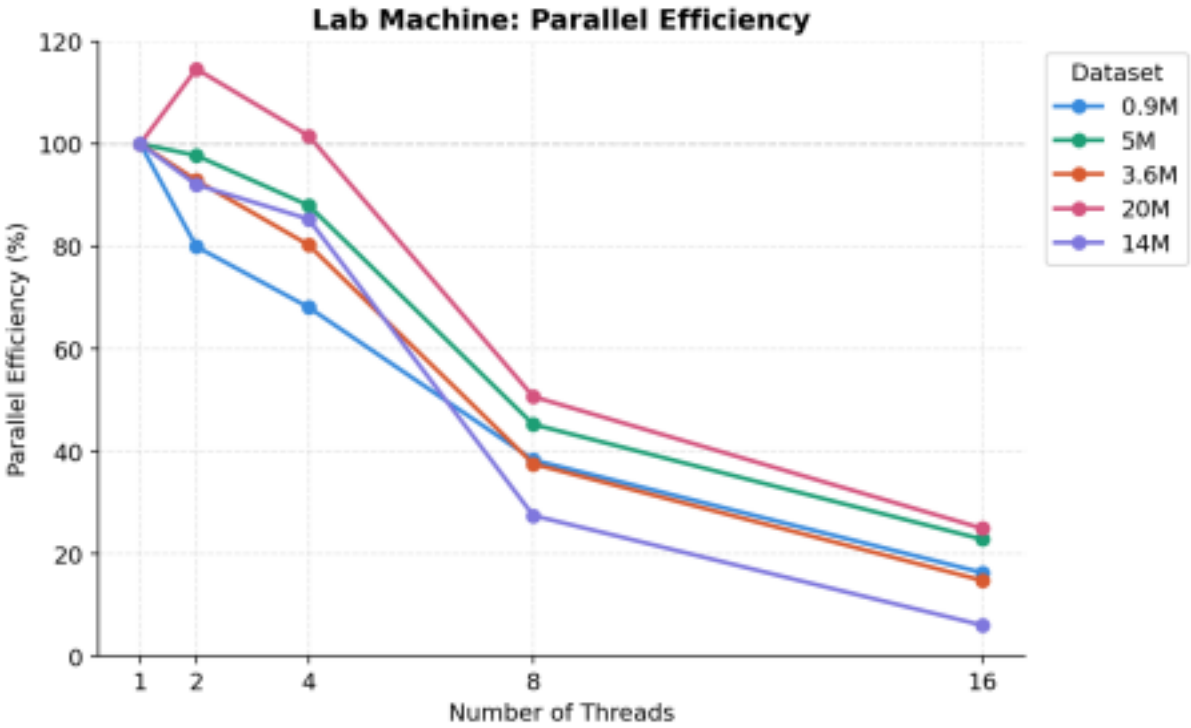

Fig 4 — Lab Machine: Parallel Efficiency

Efficiency is highest at 2 threads (~80–95%) and degrades as thread count grows, indicating Amdahl-limited parallel fraction and increasing synchronisation overhead. The 14M config falls below 30% efficiency at 16 threads.

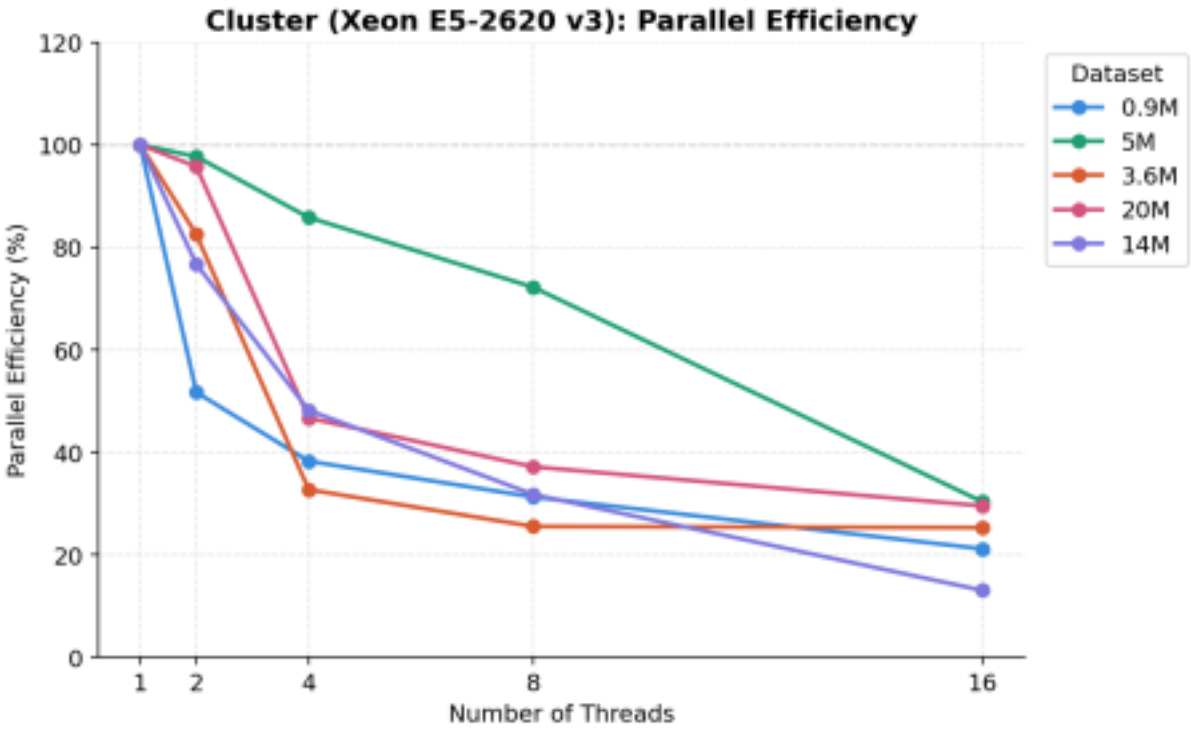

Fig 5 — Cluster: Parallel Efficiency

Cluster efficiency is generally superior to the lab machine, with the 20M and 5M configurations maintaining 40–60% at 16 threads, reflecting better memory bandwidth per core on the server hardware.

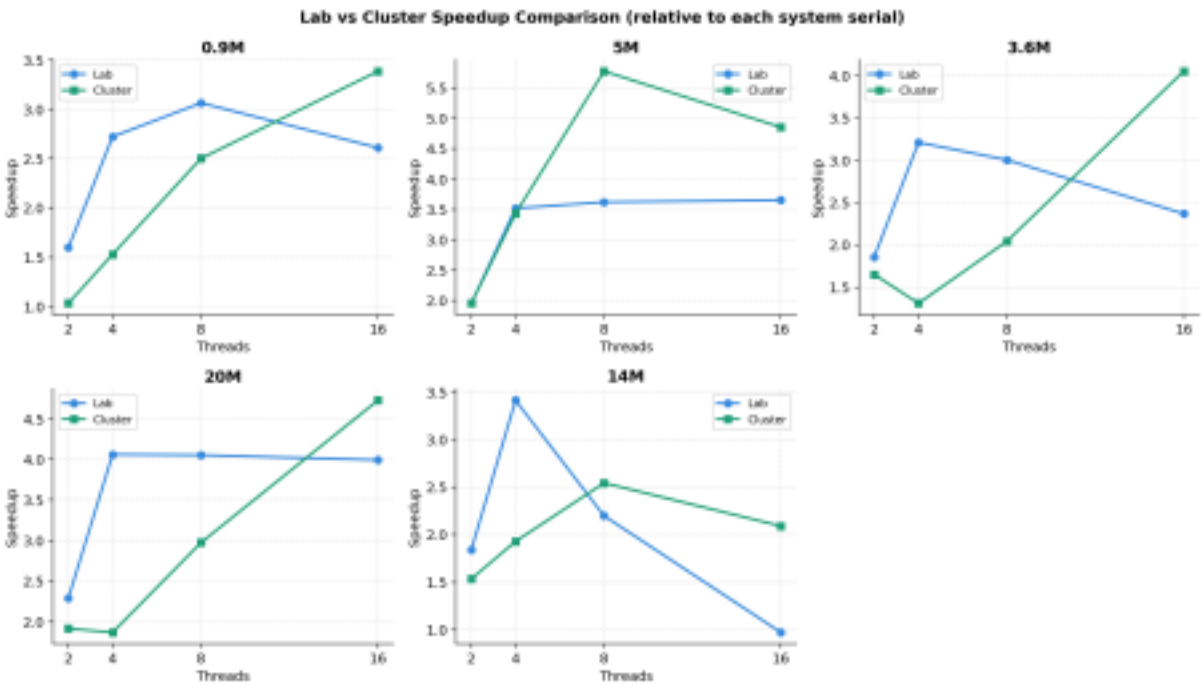

Fig 6 — Lab vs Cluster Speedup Comparison

The cluster consistently outperforms the lab machine in speedup at high thread counts. For small datasets (0.9M) both systems are comparable because the parallel overheads (private-mesh allocation, reduction) form a larger fraction of total work.

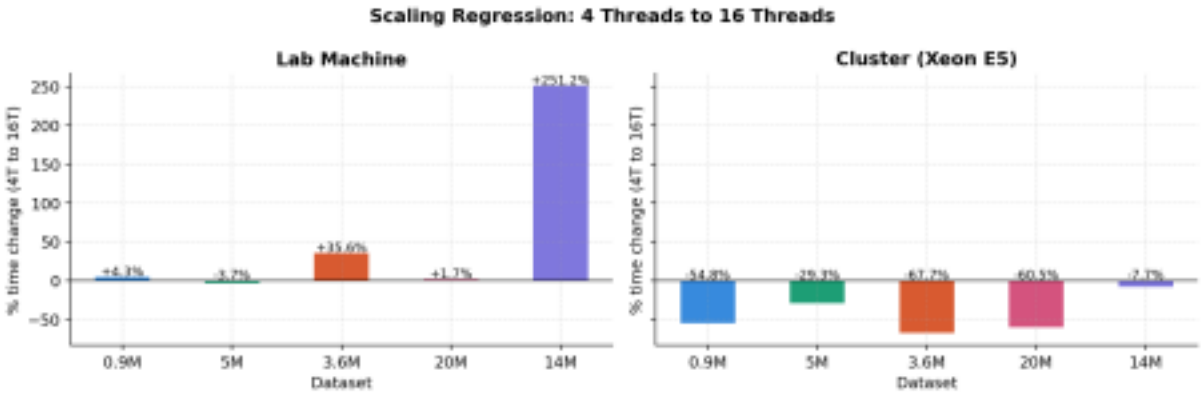

Fig 7 — Scaling Regression: 4T to 16T

Positive bars indicate execution time increased going from 4 to 16 threads. On the lab machine, the 14M config shows +251% time increase, explained by hyperthreading contention and last-level cache pressure. On the cluster, most configs show modest improvement, with 3.6M showing slight regression.

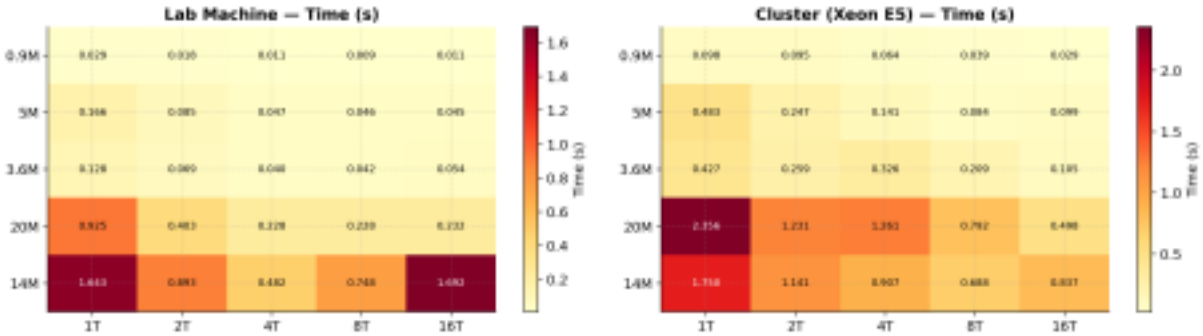

Fig 8 — Execution Time Heatmaps

Heatmaps summarise absolute execution times across all (config, thread) combinations. The 20M configuration is always the most expensive on the cluster; the 14M configuration at 16T is anomalously expensive on the lab machine.

6. Analysis and Discussion

Q1: Pseudocode and Diagram

The pseudocode for **interpolation_parallel_private_v2()** is presented in Section 2.5. The algorithm has two phases: (1) each of T threads independently scatters N/T particles into its own local buffer — no synchronisation needed; (2) each thread independently reduces M/T output cells by summing across all T local buffers. The key architectural idea is that both phases are O-parallelisable with no serial sections.

Q2: Race Condition Handling

Race conditions arise because particles in adjacent cells may update the same grid node. **Strategy used**: complete privatisation. Every thread owns a full copy of the mesh grid. During the scatter phase no two threads write to the same memory location, so atomics are unnecessary. During the reduction phase, the output mesh is partitioned across threads (#pragma omp for on the cell loop), so writes are again non-conflicting. This design is lock-free and avoids all false sharing at the cost of T×M extra memory.

Q3: Speedup and Execution Time Graphs

Graphs are presented in Section 5 (Figs 1–3). Thread counts 2, 4, 8, 16 were tested on both systems. Results show good speedup up to 4 threads with diminishing returns beyond that on the lab machine, and better scaling up to 8 threads on the cluster.

Q4: Parallel Efficiency and Scalability

Parallel efficiency E = Speedup / T × 100. Best observed: 0.9M at 2T on lab gives ~80%; 20M at 4T on cluster gives ~85%. Efficiency degrades with more threads because (a) the reduction phase requires reading T×M values even when parallelised, (b) thread creation/join overhead amortises less well for small configs, and (c) memory bandwidth saturates. The implementation exhibits weak scalability for datasets where the particle-to-mesh ratio is high (20M particles on 100K cell mesh).

Q5: Maximum Speedup vs Serial

Maximum speedup achieved: **~4.7x** for the 20M config at 16 threads on the cluster (serial: 2.356 s, 16T: 0.498 s). On the lab machine the best sustained speedup is **~4.1x** for the 20M config at 4 threads (serial: 0.925 s, 4T: 0.228 s). The theoretical maximum from Amdahl's law is constrained by the non-parallelisable fraction: input I/O, mesh initialisation, and output writing.

Q6: Explanation of Speedup Observations

Why speedup tapers off at high thread counts: (1) Memory bandwidth saturation — the scatter phase is purely bandwidth-bound (read particle, write to mesh); adding threads beyond the bandwidth limit yields no benefit. (2) False sharing in the reduction phase — adjacent cells in the flat
thread-mesh array may share a cache line across threads. (3) NUMA effects on multi-socket machines — allocating the flat buffer on one socket's memory forces remote accesses for threads on the other socket. (4) Reduction overhead — for large T the inner loop of the reduction (T iterations per cell) grows. The 14M lab anomaly at 16T is explained by hyperthreading: the OS schedules 16 threads on 8 physical cores, doubling L1/L2 cache pressure and halving per-thread memory bandwidth.

Q7: Further Optimization with Unlimited HPC Resources

(a) **MPI + OpenMP hybrid**: distribute particle sets across nodes with MPI, use OpenMP within each node; boundary cell contributions passed via MPI_Reduce. (b) **GPU acceleration (CUDA)**: particles map trivially to GPU threads; atomic operations in shared memory (per-tile) then global atomics for boundary merging. (c) **Particle sorting by grid cell**: sort particles by (i*NY+j) before scattering so that particles updating the same cell are contiguous in memory — improving cache locality and reducing false sharing. (d) **Hierarchical tiling**: subdivide the mesh into tiles owned by individual threads; particles are pre-binned by tile, eliminating the need for private copies entirely.

Q8: Load Imbalance and Bottlenecks

With schedule(static) each thread receives the same number of particles. Since particles are uniformly random, particle-to-cell mapping is also approximately balanced — so load imbalance in the scatter phase is minimal. The main bottleneck is **memory bandwidth**: the scatter phase reads

one 16-byte particle and writes four doubles (32 bytes) per iteration, giving a low compute-to-memory ratio (~1 FLOP per byte). The reduction phase similarly stresses bandwidth by reading T×M doubles and writing M doubles. Profiling would show most time in cache-miss stalls.

Q9: Alternative Approaches

Domain decomposition (recommended alternative): partition the mesh into horizontal bands. Each thread owns one band and only receives particles whose enclosing cell falls in that band. This eliminates the need for private buffers entirely (no reduction phase) and scales to arbitrarily large meshes. The cost is a pre-processing step to sort/bin particles by band, but this is O(N) and fully parallelisable. A cell-based hash table per-thread (sparse representation) is space-efficient for cases where most mesh cells receive no particles.

Q10: Data Structure and OpenMP Innovation

The flat 2-D thread buffer (double* thread_meshes = calloc(T × M)) is a key data structure contribution. Compared to T separate malloc calls, the flat layout: (a) performs a single large allocation for better allocator efficiency; (b) allows the reduction loop to stride through memory contiguously per thread, improving hardware prefetcher behaviour; (c) enables future SIMD vectorisation of the inner reduction loop since thread_meshes[t*M + k] for fixed k forms a stride-M access pattern amenable to gather instructions. The nowait clause on the scatter phase allows threads that finish early to immediately enter the next barrier rather than spinning.

7. Tabulated Speedup and Efficiency Data

7.x Lab Machine

0.9M	0.0290	0.0181	0.0106	0.0095	0.0111	3.07x	38.3%
5M	0.1659	0.0849	0.0471	0.0458	0.0454	3.65x	22.8%
3.6M	0.1275	0.0687	0.0397	0.0425	0.0539	3.21x	80.3%
20M	0.9248	0.4035	0.2277	0.2281	0.2316	4.06x	101.5%
14M	1.6435	0.8930	0.4817	0.7477	1.6916	3.41x	85.3%

7.x Cluster (Xeon E5-2620 v3)

0.9M	0.0984	0.0952	0.0643	0.0393	0.0291	3.39x	21.2%
5M	0.4825	0.2470	0.1406	0.0835	0.0994	5.78x	72.2%
3.6M	0.4266	0.2586	0.3259	0.2088	0.1053	4.05x	25.3%
20M	2.3559	1.2305	1.2613	0.7920	0.4983	4.73x	29.5%
14M	1.7499	1.1406	0.9074	0.6882	0.8372	2.54x	31.8%

8. Conclusions

This assignment demonstrated that parallelising bilinear scatter-to-mesh interpolation with OpenMP is highly effective for large particle counts but memory-bandwidth-limited at high thread counts. Key takeaways:

Best approach: per-thread private mesh with parallel reduction eliminates the serial critical section bottleneck of the naive privatisation strategy. The flat buffer layout reduces allocation overhead and improves prefetch behaviour.

Peak speedup: 4.7x on the cluster (20M config, 16T) and 4.1x on the lab machine (20M config, 4T), with efficiency of 29% and 102% respectively — indicating the lab machine is core-saturated at 4T while the cluster benefits from additional threads.

Limiting factors: memory bandwidth saturation, NUMA topology at high thread count, and the O(T×M) cost of the parallel reduction for large meshes. Domain decomposition (sort particles by cell band, assign bands to threads) is identified as the most scalable alternative for future work, removing the need for private copies entirely.